

CloudVerse + CloudLab

Full Installation Guide — Postgres, pgAdmin, Redis, Docker, Go, and the App

Everything needed to go from a bare Linux machine to a running CloudVerse + CloudLab stack: installing every prerequisite (Node.js, PostgreSQL, pgAdmin, Redis, Docker Engine + containerd, and a Go toolchain), then the application's own one-time setup, how to start it, how to verify each piece is actually working, and troubleshooting for the failures you're most likely to hit.

Prepared for	Sai Rathish
Organization	Prodapt-AID Practice
Covers	OS prerequisites + application install, run, and verify steps
Target OS	Ubuntu/Debian Linux (notes included for macOS, Fedora/RHEL, Arch, WSL2)
Date	September 11, 2026

How to use this guide: work through Section 1 top to bottom in a terminal on your Linux environment (a native Linux machine, a Linux VM/app on your laptop, or WSL2 on Windows). Each block of commands is meant to be pasted as-is; adjust only where a comment tells you to. Section 2 covers the application itself once the prerequisites are in place.

1. Installing the prerequisites (Ubuntu / Debian)

The application needs five things on the machine before it will run: Node.js, PostgreSQL (plus pgAdmin as an optional GUI), Redis, a Docker daemon with its own containerd, and a Go toolchain (used once, to compile five small local container images). Install them in this order.

1.1 Update the system

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg lsb-release git build-essential
```

1.2 Node.js 20+

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

node -v    # expect v20.x or later
npm -v
```

1.3 PostgreSQL + pgAdmin

PostgreSQL is the real database the application stores every project, resource, and secret in. pgAdmin is optional — a graphical client for browsing that same database — install it if you want a UI for inspecting tables instead of using psql on the command line.

Install PostgreSQL server:

```
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql

# Set a password for the postgres superuser (used in your .env DATABASE_URL):
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'yourpassword';"

# Confirm it's up:
sudo -u postgres psql -c "SELECT version();"
```

Install pgAdmin4 (optional GUI):

```
curl -fsSL https://www.pgadmin.org/static/packages_pgadmin_org.pub | \
  sudo gpg --dearmor -o /usr/share/keyrings/packages-pgadmin-org.gpg

sudo sh -c 'echo "deb [signed-by=/usr/share/keyrings/packages-pgadmin-org.gpg] \
  https://ftp.postgresql.org/pub/pgadmin/pgadmin4/apt/$(lsb_release -cs) pgadmin4 main" \
  > /etc/apt/sources.list.d/pgadmin4.list'

sudo apt update
sudo apt install -y pgadmin4-desktop    # a native window on a Linux desktop
# -- or, if you're on a headless/server box with no desktop --
sudo apt install -y pgadmin4-web        # browser-based, served over Apache
```

Using pgAdmin with this app. Register a new server in pgAdmin pointing at localhost, port 5432, user postgres, and the password you set above. Once the app's migrations have run (Section 2), you'll see a cloudlab database with tables for projects, resources, secrets, and audit logs — useful for inspecting state directly while testing.

1.4 Redis

```
sudo apt install -y redis-server
sudo systemctl enable --now redis-server

redis-cli ping    # expect: PONG
```

1.5 Docker Engine + containerd

The worker process provisions and tears down real containers through Docker, so a working Docker daemon is required — this isn't optional or simulated. Docker Engine on Linux always runs on top of containerd; the official install below sets up both together.

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
  stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compo
se-plugin

sudo systemctl enable --now containerd
sudo systemctl enable --now docker

# Let your user run docker without sudo (log out and back in after this):
sudo usermod -aG docker $USER

# Verify:
docker run hello-world
```

Order matters here. dockerd depends on containerd already running and reachable at /run/containerd/containerd.sock. The systemd units above start containerd first for exactly this reason — if you ever install Docker a different way and "docker" commands hang or fail to connect, check systemctl status containerd before assuming Docker itself is broken.

1.6 Go toolchain

Needed once, to compile the five small local-only container images (guest, registry, bucket, dns, load balancer) that back the application's real provisioning. Each is a static Go binary assembled into a FROM scratch image

with zero external registry pulls.

```
curl -LO https://go.dev/dl/go1.22.5.linux-amd64.tar.gz
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.22.5.linux-amd64.tar.gz

echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc

go version    # expect go1.22.x
```

1.7 Verify everything before moving on

```
node -v && npm -v
sudo -u postgres psql -c "SELECT version();"
redis-cli ping
docker run hello-world
go version
```

If all five commands above succeed, every prerequisite is installed correctly and you're ready for the application setup in Section 2.

2. Installing and running the application

CloudVerse and CloudLab are one npm-workspace monorepo with three long-running processes (a Fastify API + the original static console on :4000, a BullMQ worker, and the CloudVerse Next.js console on :3001) sitting in front of the real PostgreSQL, Redis, and Docker daemon set up above. Nothing here is mocked — every resource type is either genuinely provisioned on your machine, or, where noted "Simulated" elsewhere in the app, honestly labeled as a teaching-only exercise.

2.1 One-time setup

- 1 Unzip the application source and, from its root folder, install dependencies for every workspace (apps/api, apps/worker, apps/console, packages/shared) in one command.
- 2 Copy the two example environment files to their real names and fill in your own values.
- 3 Generate your own Secrets-Manager master key rather than leaving it blank or reusing an example.
- 4 Create the Postgres database and run migrations.
- 5 Build the five local container images the platform provisions from.

```
npm install

cp .env.example .env
cp apps/console/.env.local.example apps/console/.env.local
# edit .env: DATABASE_URL, REDIS_URL, DOCKER_SOCKET if any aren't at the defaults

# Generate a real 32-byte base64 key for SECRETS_MASTER_KEY in .env:
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

createdb cloudlab # or: sudo -u postgres createdb cloudlab
npm run db:migrate

# Build the five local-only images (each is FROM scratch, no external pulls):
bash infra/guest-image/build.sh # backs Compute instances, Cloud Run, MIG members
bash infra/registry-image/build.sh # backs the Registry / Artifact Registry feature
bash infra/bucket-image/build.sh # backs Cloud Storage buckets
bash infra/dns-image/build.sh # backs Cloud DNS / DNS records
bash infra/lb-image/build.sh # backs Load Balancers
```

2.2 Running it

Three long-running processes, each needs its own terminal window/tab (or a process manager like pm2 or tmux) — closing any one of them takes that piece of the stack down:

```
# Terminal 1
npm run dev:api # Fastify API + the original static console, on :4000

# Terminal 2
npm run dev:worker # BullMQ worker -- actually provisions/tears down Docker resources

# Terminal 3
cd apps/console && npm run dev # CloudVerse DevOps Academy console, on :3001
```

2.3 Verifying it works

- 1 Open <http://localhost:4000> for the original CloudLab console, and <http://localhost:3001> for the CloudVerse DevOps Academy console. Both talk to the same API and the same database — a resource created in one is visible from the other.
- 2 In pgAdmin (or psql), connect to the cloudlab database and confirm tables like projects, resources, and secrets now exist after the migration step.
- 3 Create a real resource (e.g. a VM or VPC) from either console, then run "docker ps" in a terminal -- you should see a real container appear for it, confirming the worker is actually talking to Docker rather than just updating a database row.

```
docker ps --filter label=cloudlab.k8sRole=pod # real k8s-simulated pods, if any are running
psql -h localhost -U postgres -d cloudlab -c "select count(*) from resources;"
```

2.4 Troubleshooting

Docker daemon won't start. dockerd depends on containerd already running and communicating over /run/containerd/containerd.sock. If "systemctl start docker" fails silently, check "systemctl status containerd" first (start it, then start dockerd) rather than assuming Docker itself is broken.

A real VPC network fails to create with "overlaps with network...". No real IP-address-management/allocator exists yet, so a freshly-suggested CIDR occasionally collides with a leftover network from an earlier session in a long-lived environment. This is a documented, expected limitation, not a bug — just pick (or regenerate) a different CIDR and retry.

"permission denied" running docker commands without sudo. You were added to the docker group in Step 1.5 but haven't logged out and back in yet (or started a new shell session) for that group membership to take effect. Log out/in, or run "newgrp docker" in the current shell.

Port already in use (4000 / 3001 / 5432 / 6379). Something else on the machine is already bound to one of these ports. Find it with "sudo lsof -i :4000" (swap the port number) and either stop that process or change the app's port/URL in .env.

3. Notes for non-Ubuntu systems

Every prerequisite above is cross-platform -- Node.js, PostgreSQL, pgAdmin, Redis, Docker, and Go all run the same way on any Linux distribution, macOS, or Windows via WSL2. Only the package-manager commands in Section 1 are Ubuntu/Debian-specific (apt). Swap them for your platform's equivalent:

Tool	Fedora / RHEL (dnf)	Arch (pacman)	macOS (brew)
Node.js 20+	<code>dnf module install nodejs:20</code>	<code>pacman -S nodejs npm</code>	<code>brew install node@20</code>
PostgreSQL	<code>dnf install postgresql-server</code>	<code>pacman -S postgresql</code>	<code>brew install postgresql@16</code>
pgAdmin	<code>dnf install pgadmin4</code>	<code>pacman -S pgadmin4</code>	<code>brew install --cask pgadmin4</code>
Redis	<code>dnf install redis</code>	<code>pacman -S redis</code>	<code>brew install redis</code>
Docker	<code>dnf install docker-ce ...</code>	<code>pacman -S docker</code>	Docker Desktop for Mac
Go	<code>dnf install golang</code>	<code>pacman -S go</code>	<code>brew install go</code>

On Windows, install WSL2 first ("wsl --install" from an admin PowerShell), then treat the Ubuntu WSL environment it gives you exactly like Section 1 above. On WSL2, Docker Desktop for Windows (with WSL2 integration enabled) is usually simpler than installing Docker Engine inside WSL directly.

3.1 The one Ubuntu-specific line inside the app itself

Everywhere else in this guide, the app's own build scripts are fully cross-platform. The single exception is a fallback inside `infra/guest-image/build.sh`, and it only runs if `busybox` isn't already on your `PATH`:

```
BUSYBOX_BIN="$(command -v busybox || echo /usr/bin/busybox)"
if [ ! -x "$BUSYBOX_BIN" ]; then
  echo "busybox not found -- installing busybox-static via apt" >&2
  apt-get update -y && apt-get install -y busybox-static
  BUSYBOX_BIN=/usr/bin/busybox
fi
```

On a non-Debian system, just install `busybox` yourself first (`dnf install busybox`, `pacman -S busybox`, `brew install busybox`) and the script picks it up without ever touching `apt`.